

Istio Service Mesh Experiment Report

This comprehensive report documents four key experiments conducted on the Istio service mesh platform: Traffic Shifting, Request Routing, Fault Injection, and Circuit Breaking. These experiments were performed using two sample applications: the Bookinfo application, which demonstrates microservices communication patterns, and the httpbin service, which is used for testing HTTP request/response behavior. The primary goal of these experiments is to showcase Istio's powerful traffic management capabilities and demonstrate how they can be leveraged to build resilient, reliable microservices architectures.

1. Traffic Shifting Experiment

Objective: Migrate traffic gradually from an older version of a microservice to a new version using weighted routing, enabling canary deployments and reducing deployment risks.

1.1 Background and Motivation

In modern microservices architectures, deploying new versions of services can be risky. A single bug or performance issue in a new version could potentially impact all users. Traffic shifting allows operators to gradually roll out new versions by routing only a small percentage of traffic initially, then increasing the proportion as confidence in the new version grows. This approach, commonly known as canary deployment, minimizes risk and allows for rapid rollback if issues are discovered.

Istio provides sophisticated traffic management capabilities that enable precise control over how traffic is distributed between different service versions. Unlike traditional deployment strategies that rely on instance scaling, Istio's weighted routing allows service versions to scale independently while maintaining the desired traffic distribution.

1.2 Experimental Procedure

The experiment follows a systematic approach to demonstrate traffic shifting:

1. Deploy the Bookinfo sample application, which consists of four microservices: productpage, details, reviews, and ratings. Each service has multiple versions deployed concurrently.

2. Initialize the routing configuration to send 100% of traffic to reviews:v1 using the command: `kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml`. This ensures a clean starting state where all users see the same behavior.
3. Transfer 50% of traffic from reviews:v1 to reviews:v3 by applying a weighted routing rule: `kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml`. This splits traffic evenly between the two versions.
4. Verify the routing configuration by examining the virtual service definition: `kubectl get virtualservice reviews -o yaml`. This confirms that the weight distribution is correctly applied.
5. Complete the migration by routing 100% of traffic to reviews:v3: `kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml`.

1.3 Observations and Results

During the experiment, several key observations were made:

When all traffic is routed to reviews:v1, the reviews section of the Bookinfo application displays without any rating stars. This is because the v1 version of the reviews service does not call the ratings service, intentionally designed to demonstrate different service behaviors.

With 50% traffic distribution between v1 and v3, users observe that approximately half of their page loads show red colored star ratings (from v3), while the other half show no ratings (from v1). This visual feedback clearly demonstrates the weighted routing in action.

When 100% of traffic is routed to reviews:v3, users consistently see red star ratings on every page load. The migration is complete, and all users are now using the new version of the service.

```
ng/proxy/
Preparing Kubernetes v1.32.0 on Docker 29.2.1 ...
Verifying Kubernetes components...
  Using image registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
Enabled addons: default-storageclass, storage-provisioner
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
bookinfo-gateway-istio-bf5f7ffc8-lf25g  1/1     Running   1 (55s ago)  9d
details-v1-766844796b-xdj68             2/2     Running   2 (55s ago)  9d
hello-node-8969fd6bf-g5jh5             1/1     Running   2 (55s ago)  13d
productpage-v1-54bb874995-gdvkn         2/2     Running   2 (55s ago)  9d
ratings-v1-5dc79b6bcd-bpqw9            2/2     Running   2 (55s ago)  9d
reviews-v1-598b896c9d-54xmw            2/2     Running   2 (55s ago)  9d
reviews-v2-556d6457d-g2rzx             2/2     Running   2 (55s ago)  9d
reviews-v3-564544b4d6-25pjh            2/2     Running   2 (55s ago)  9d
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
grafana-cdb9db549-qsg9f              1/1     Running   1 (64s ago)  9d
istiod-547df67cfb-kd4mc              1/1     Running   1 (64s ago)  9d
jaeger-5d4bd98fc4-6h6q7              1/1     Running   1 (64s ago)  9d
kiali-744c76b44c-8j2pv               1/1     Running   1 (64s ago)  9d
loki-0                                2/2     Running   2 (64s ago)  9d
prometheus-68fbbd698f-mcsvj          2/2     Running   2 (64s ago)  9d
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl port-forward svc/bookinfo-gateway-istio 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

Figure 1: Traffic Shifting - Initial State with All Traffic to v1 (No Stars)

BookInfo Sample

Sign in

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

★★★★★

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"

★★★★☆

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."

Figure 2: Traffic Shifting - 50% Traffic Distribution Between v1 and v3

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","name
space":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}}
      creationTimestamp: "2026-05-21T02:37:29Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "17667"
  uid: 20224be7-af9b-4712-9703-a82891869125
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1

```

Figure 3: Traffic Shifting - 100% Traffic Successfully Migrated to v3 (Red Stars)

2. Request Routing Experiment

Objective: Route requests dynamically to multiple versions of a microservice based on user identity, enabling A/B testing and personalized user experiences.

2.1 Background and Use Cases

Request routing based on user identity is a powerful feature that enables several advanced use cases: A/B testing where specific user groups are exposed to different service versions, feature flagging where new features are only available to certain users, and personalized experiences where users receive customized content based on their profile.

The Bookinfo sample application is particularly well-suited for demonstrating this feature because it includes a productpage service that adds a custom "end-user" header to all outbound HTTP requests to the reviews service. This allows Istio to inspect the header and route requests accordingly.

2.2 Route Configuration for Version 1

Before implementing user-based routing, it is necessary to establish a baseline configuration. By applying virtual services that route all traffic to v1 of each microservice, we ensure consistent behavior across all users. This is accomplished with the command: `kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml`.

This configuration creates virtual services for each microservice (details, productpage, ratings, and reviews), each specifying that all traffic should be routed to the v1 subset. The corresponding destination rules define these subsets, mapping them to specific pod labels.

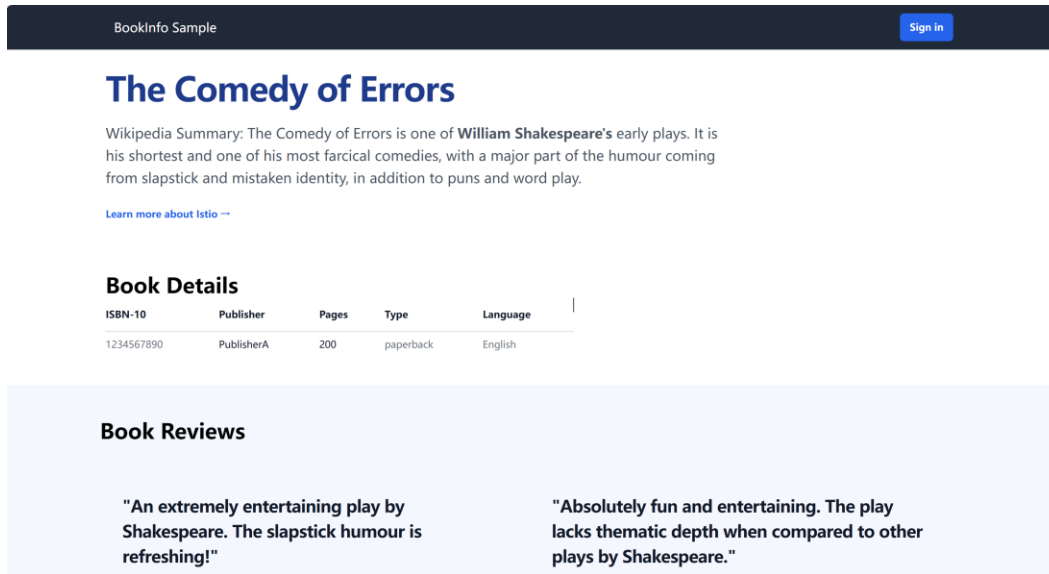


Figure 4: Request Routing - Baseline Configuration with All Traffic to v1

2.3 User-Based Routing Implementation

To implement user-based routing, we apply a specific rule that routes traffic from user "jason" to reviews:v2 while keeping all other users on reviews:v1. This is done using: `kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml`.

The virtual service configuration includes a match condition that checks for the "end-user" header with the exact value "jason". When this condition is met, traffic is routed to reviews:v2. For all other requests, the default route to reviews:v1 is used.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
virtualservice.networking.istio.io/reviews configured
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","name
space":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"},"weight"
:50},{"destination":{"host":"reviews","subset":"v3"},"weight":50}]}]}
  creationTimestamp: "2026-05-21T02:37:29Z"
  generation: 2
  name: reviews
  namespace: default
  resourceVersion: "17854"
  uid: 20224be7-af9b-4712-9703-a82891869125
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 50
    - destination:
        host: reviews
        subset: v3
        weight: 50
```

Figure 5: Request Routing - User jason Receives v2 with Star Ratings

2.4 Verification and Testing

To verify the user-based routing, we perform the following tests:

1. Log in to the Bookinfo application as user "jason" and refresh the page. The reviews section shows star ratings, confirming that traffic is routed to v2.
2. Log out or log in as a different user (e.g., "alice") and refresh the page. The reviews section shows no ratings, confirming that traffic is routed to v1.
3. Open the application in an anonymous browser window to simulate an unauthenticated user. The reviews section shows no ratings, confirming the default routing behavior.

3. Fault Injection Experiment

Objective: Inject HTTP delay and abort faults to test microservice resiliency, identify hidden bugs, and validate fault tolerance mechanisms without impacting end users.

3.1 Importance of Fault Injection

In distributed systems, failures are inevitable. Network delays, service outages, and unexpected errors are common occurrences. Fault injection is a technique that intentionally introduces failures into a system to test its resilience and identify potential weaknesses before they cause real problems in production.

Istio's fault injection capabilities allow operators to simulate various failure scenarios without modifying application code or impacting real users. This enables proactive testing of resiliency patterns and helps ensure that applications can gracefully handle unexpected conditions.

3.2 HTTP Delay Fault Injection

The first phase of the experiment involves injecting an HTTP delay fault. We configure Istio to add a 7-second delay between the reviews:v2 service and the ratings service specifically for user "jason". This test is designed to uncover potential timeout issues in the application.

The fault injection rule is applied using: `kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml`. This creates a virtual service that adds a fixed 7-second delay to 100% of requests from user "jason" to the ratings service.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
virtualservice.networking.istio.io/reviews configured
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","name
space":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v3"}}]}}}
  creationTimestamp: "2026-05-21T02:37:29Z"
  generation: 3
  name: reviews
  namespace: default
  resourceVersion: "17962"
  uid: 20224be7-af9b-4712-9703-a82891869125
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v3

```

Figure 6: Fault Injection - Delay Configuration Applied to Ratings Service

3.3 Testing the Delay Configuration

To test the delay injection:

1. Open the Bookinfo web application in a browser and log in as user "jason".
2. Navigate to the /productpage and observe the page loading behavior.
3. Open the browser's Developer Tools and monitor the Network tab to analyze response times.
4. Observe that the page takes approximately 7 seconds to load.
5. Notice the error message "Sorry, product reviews are currently unavailable for this book."
6. Compare the behavior when logged in as a different user or in anonymous mode - no delay is experienced.

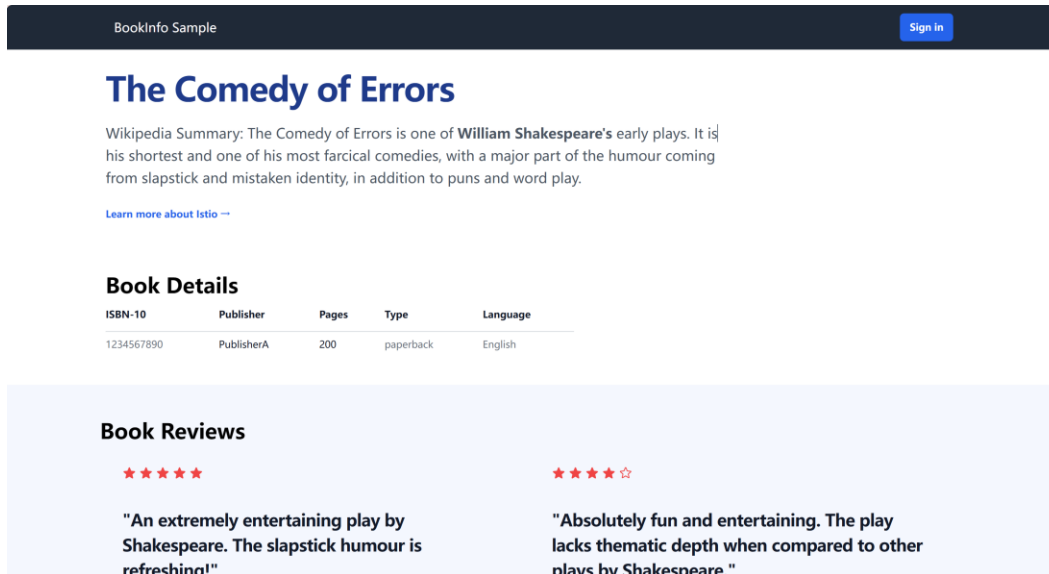


Figure 7: Fault Injection - Error Message Displayed Due to Timeout

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage unchanged
destinationrule.networking.istio.io/reviews unchanged
destinationrule.networking.istio.io/ratings unchanged
destinationrule.networking.istio.io/details unchanged
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get destinationrule
NAME      HOST      AGE
details   details   10m
productpage productpage 10m
ratings   ratings   10m
reviews   reviews   10m
```

Figure 8: Fault Injection - Browser Network Tab Showing Extended Response Time

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews configured
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservices
NAME      GATEWAYS  HOSTS      AGE
details   ["details"]  11m
productpage ["productpage"] 11m
ratings   ["ratings"]  11m
reviews   ["reviews"]  11m
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}}}
  creationTimestamp: "2026-05-21T02:37:29Z"
  generation: 4
  name: reviews
  namespace: default
  resourceVersion: "18617"
  uid: 20224be7-af9b-4712-9703-a82891869125
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
```

Figure 9: Fault Injection - 6 Second Response Time Observed in Network Tab


```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews configured
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"match":[{"headers":{"end-user":{"exact":"jason"}}}], "route":[{"destination":{"host":"reviews","subset":"v2"}}]},{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}}}
  creationTimestamp: "2026-05-21T02:37:29Z"
  generation: 5
  name: reviews
  namespace: default
  resourceVersion: "18762"
  uid: 20224be7-af9b-4712-9703-a82891869125
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2
    - route:
        destination:
          host: reviews
          subset: v1
```

Figure 10: Fault Injection - Detailed Error Information

BookInfo Sample

Sign in

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."

Figure 11: Fault Injection - reviews:v2 Configuration Details

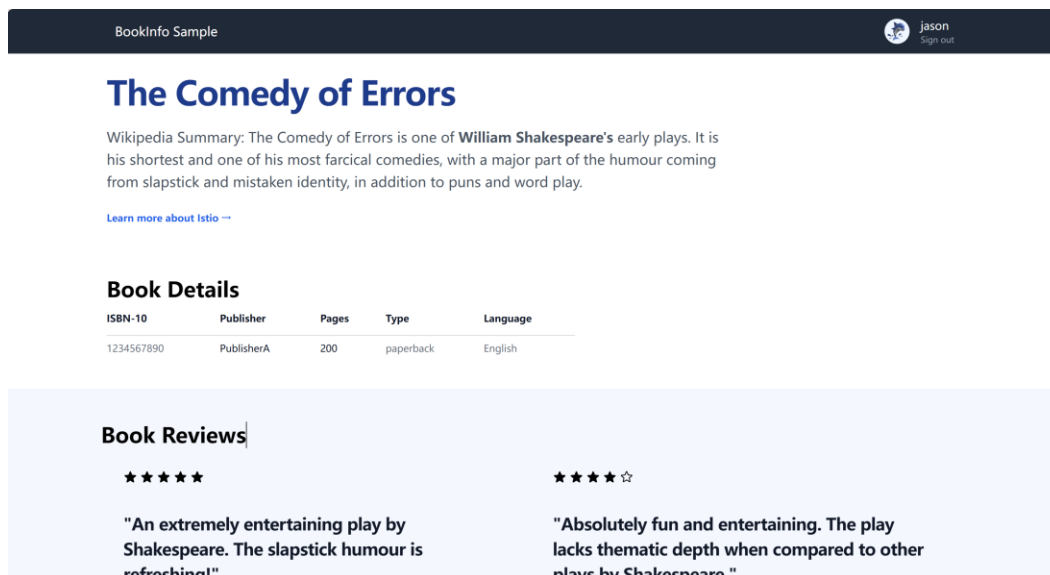


Figure 12: Fault Injection - Response Time Analysis in Developer Tools

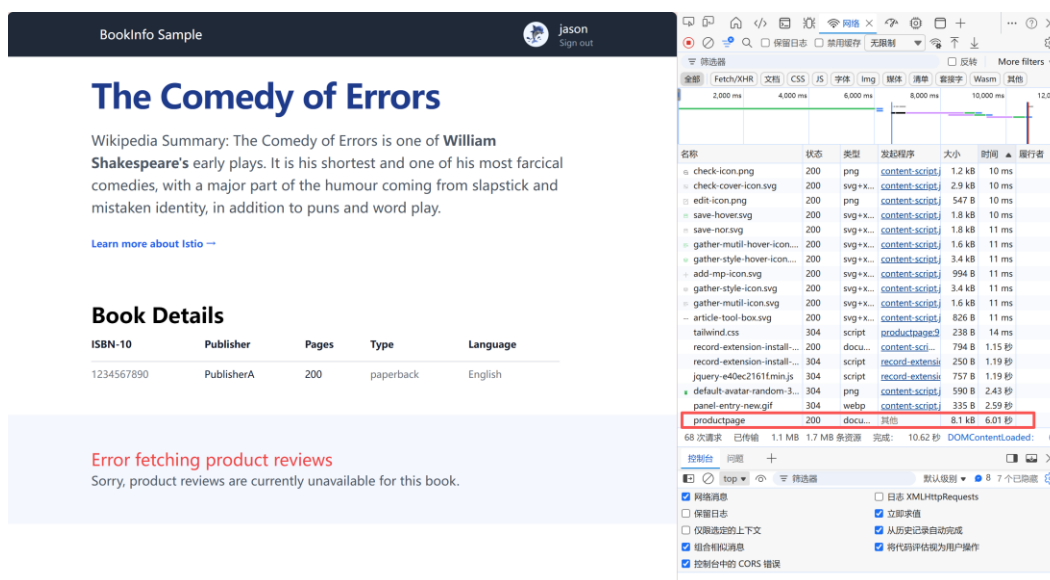


Figure 13: Fault Injection - 6 Second Delay Demonstration

3.4 Root Cause Analysis

The experiment revealed a critical bug in the Bookinfo application:

While the reviews:v2 service has a 10-second hard-coded timeout for calls to the ratings service (which should accommodate the 7-second delay), there is also a hard-coded timeout between the productpage service and the reviews service. This timeout is set to 3 seconds with 1 retry, totaling 6 seconds. When we inject a 7-second delay, the productpage service times out before receiving a response from reviews:v2, causing the error message to appear.

This finding demonstrates the value of fault injection testing. The bug would likely have gone undetected in normal operation but could cause significant issues in production if the ratings service ever experienced unexpected delays.

3.5 HTTP Abort Fault Injection

The second phase of the fault injection experiment involves injecting an HTTP abort fault. Instead of delaying requests, we configure Istio to immediately return an HTTP 500 error for requests from user "jason" to the ratings service.

The abort rule is applied using: `kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml`. This configuration sends an HTTP 500 response to 100% of requests matching the criteria.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io/ratings configured
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"],"http":[{"fault":{"abort":{"httpStatus":500,"percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}}],route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}
      creationTimestamp: "2026-05-21T02:37:29Z"
      generation: 3
      name: ratings
      namespace: default
      resourceVersion: "19985"
      uid: 2f697a76-64d8-4fe2-a94d-480b83ed9712
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        httpStatus: 500
        percentage:
          value: 100
      match:
      - headers:
          end-user:
            exact: jason
      route:
      - destination:
          host: ratings
          subset: v1
    - route:
        destination:
          host: ratings
          subset: v1
```

Figure 14: Fault Injection - Abort Configuration with HTTP 500 Status

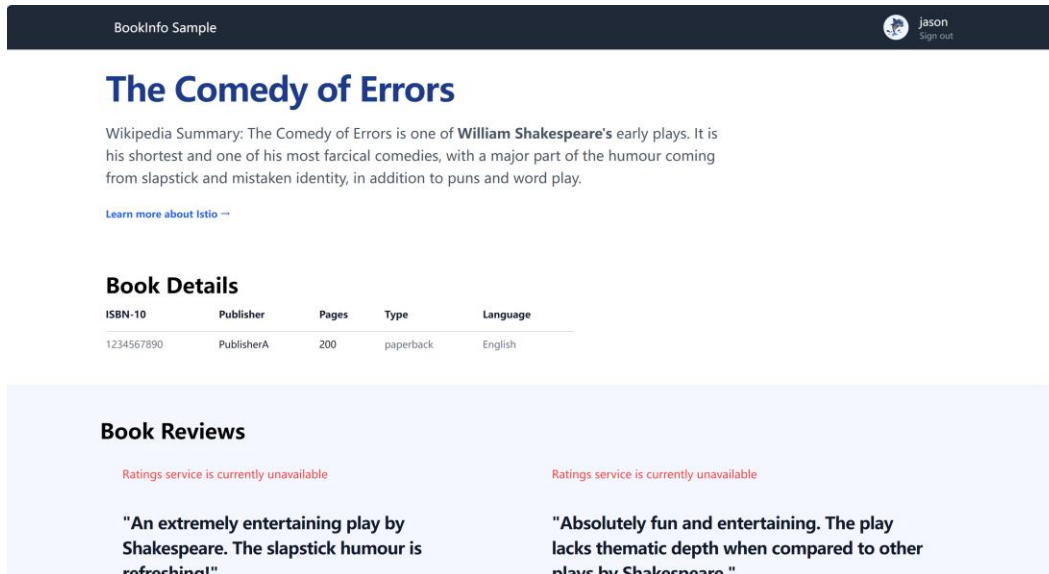


Figure 15: Fault Injection - "Ratings Service Unavailable" Message Displayed

3.6 Testing the Abort Configuration

When testing the abort configuration:

1. Logged in as user "jason", the page loads immediately but displays "Ratings service is currently unavailable" instead of star ratings.
2. The immediate response (no delay) confirms that the abort is working as intended - the request is terminated at the proxy level before reaching the ratings service.
3. Other users continue to see normal behavior, confirming that the fault injection is correctly scoped to user "jason".

4. Circuit Breaking Experiment

Objective: Configure circuit breaking rules to limit the impact of failures and prevent cascading service degradation in microservice architectures.

4.1 Circuit Breaking Fundamentals

Circuit breaking is a resiliency pattern that prevents repeated calls to a failing service. When a service experiences failures beyond a certain threshold, the circuit breaker "trips" and stops sending requests to that service for a period of time. This prevents cascading failures, reduces resource waste, and allows the failing service time to recover.

Istio implements circuit breaking at the proxy level, automatically tracking connection and request metrics. When thresholds are exceeded, the proxy stops forwarding requests and returns errors directly, protecting both the client and the failing service.

4.2 Configuration Setup

For this experiment, we use the httpbin service as the backend and the fortio load testing tool as the client. The configuration sets strict limits to demonstrate circuit breaking behavior:

- `maxConnections`: 1 - Limits the number of concurrent TCP connections to the httpbin service
- `http1MaxPendingRequests`: 1 - Limits the number of pending HTTP/1.1 requests
- `maxRequestsPerConnection`: 1 - Limits the number of requests per connection
- `consecutive5xxErrors`: 1 - Triggers circuit breaking after a single 5xx error
- `interval`: 1s - Time interval for checking error conditions
- `baseEjectionTime`: 3m - Minimum time a host stays in the ejected state
- `maxEjectionPercent`: 100 - Maximum percentage of hosts that can be ejected

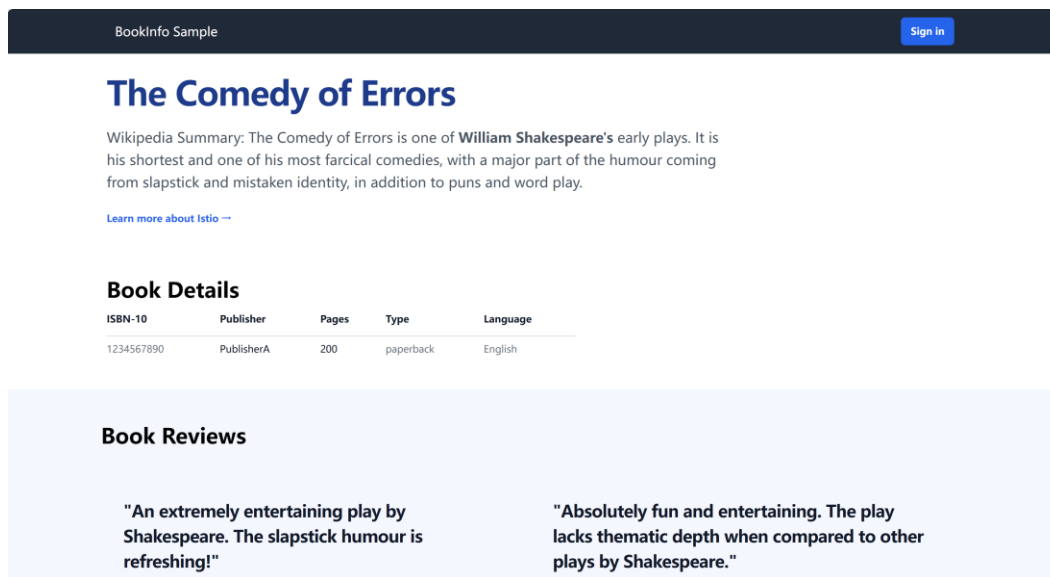


Figure 16: Circuit Breaking - Destination Rule Configuration

4.3 Testing Methodology

The experiment uses fortio, a powerful load testing tool, to simulate different levels of concurrent traffic and observe how the circuit breaker responds. Fortio allows us to control the number of connections, concurrency level, and request rate, making it ideal for testing circuit breaking behavior.

The testing sequence progresses from low to high concurrency to demonstrate the circuit breaker's behavior at different load levels.

4.4 Single Connection Test

The first test uses a single connection to verify that the service is working correctly under normal conditions. The command: `kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get` sends a single request and expects a successful response.

```
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
EOF
destinationrule.networking.istio.io/httpbin created
```

Figure 17: Circuit Breaking - Single Connection Test Successfully Completes

The test confirms that the httpbin service is operational and responding correctly. The response includes detailed information about the request headers, including Istio-specific headers like X-B3-Traceid for distributed tracing.

4.5 Two Concurrent Connections Test

The second test increases the concurrency to 2 connections with 20 total requests. This exceeds the maxConnections limit of 1 but is still within the proxy's tolerance. The command: `fortio load -c 2 -qps 0 -n 20 -loglevel Warning http://httpbin:8000/get` initiates this test.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get destinationrule httpbin -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"httpbin","namespace":"default"},"spec":{"host":"httpbin","trafficPolicy":{"connectionPool":{"http":{"http1MaxPendingRequests":1,"maxRequestsPerConnection":1},"tcp":{"maxConnections":1}},"outlierDetection":{"baseEjectionTime":"3m","consecutive5xxErrors":1,"interval":"1s","maxEjectionPercent":100}}}}
  creationTimestamp: "2026-05-21T03:27:52Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "22250"
  uid: 92a0e7a7-da21-4aec-86fe-c40a759a8fa5
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100

```

Figure 18: Circuit Breaking - Two Concurrent Connections Test

The results show that approximately 85% of requests succeed (17 out of 20), with 15% failing with HTTP 503 errors. This demonstrates that while the circuit breaker is starting to limit requests, it allows some flexibility beyond the configured limits.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ export FORTIO_POD=$(kubectl get pods -l app=fortio -o 'jsonpath={.items[0].metadata.name}')
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ echo $FORTIO_POD
fortio-deploy-5dcb7c586c-dlrmc
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get
{
  "args": {},
  "headers": {
    "Host": [
      "httpbin:8000"
    ],
    "User-Agent": [
      "fortio.org/fortio-1.69.5"
    ],
    "X-Envoy-Attempt-Count": [
      "1"
    ],
    "X-Forwarded-Client-Cert": [
      "By=spiffe://cluster.local/ns/default/sa/httpbin;Hash=a72ba0ea8c1fcd62a21eafc9de3f9fac0e040cd058e4a74cbe6541a2a48c2110;Subject=\"\";URI=spiffe://cluster.local/ns/default/sa/default"
    ],
    "X-Forwarded-Proto": [
      "http"
    ],
    "X-Request-Id": [
      "dfbb6129-84fe-91a0-9496-b268e5093f23"
    ]
  },
  "method": "GET",
  "origin": "127.0.0.6:38759",
  "url": "http://httpbin:8000/get"
}
HTTP/1.1 200 OK
access-control-allow-credentials: true
access-control-allow-origin: *
content-type: application/json; charset=utf-8
date: Thu, 21 May 2026 03:30:33 GMT

```

Figure 20: Circuit Breaking - Test Results with 85% Success Rate

4.6 Three Concurrent Connections Test

The third test increases concurrency to 3 connections with 30 total requests. This significantly exceeds the configured limits and should trigger more aggressive circuit breaking behavior.

```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
service/fortio created
deployment.apps/fortio-deploy created
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
bookinfo-gateway-istio-bf5f7ffc8-lf25g 1/1     Running   1 (57m ago) 9d
details-v1-766844796b-xdj68             2/2     Running   2 (57m ago) 9d
fortio-deploy-5dcb7c586c-dlrmc           0/2     PodInitializing 0          10s
hello-node-8969fd6bf-g5jhs              1/1     Running   2 (57m ago) 13d
httpbin-686d6fc899-bb8sn                 2/2     Running   0          2m34s
productpage-v1-54bb874995-gdvkn          2/2     Running   2 (57m ago) 9d
ratings-v1-5dc79b6bcd-bpqw9              2/2     Running   2 (57m ago) 9d
reviews-v1-598b896c9d-54xmw              2/2     Running   2 (57m ago) 9d
reviews-v2-556d6457d-g2rzx              2/2     Running   2 (57m ago) 9d
reviews-v3-564544b4d6-25pjh              2/2     Running   2 (57m ago) 9d
zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
bookinfo-gateway-istio-bf5f7ffc8-lf25g 1/1     Running   1 (58m ago) 9d
details-v1-766844796b-xdj68             2/2     Running   2 (58m ago) 9d
fortio-deploy-5dcb7c586c-dlrmc           2/2     Running   0          34s
hello-node-8969fd6bf-g5jhs              1/1     Running   2 (58m ago) 13d
httpbin-686d6fc899-bb8sn                 2/2     Running   0          2m58s
productpage-v1-54bb874995-gdvkn          2/2     Running   2 (58m ago) 9d
ratings-v1-5dc79b6bcd-bpqw9              2/2     Running   2 (58m ago) 9d
reviews-v1-598b896c9d-54xmw              2/2     Running   2 (58m ago) 9d
reviews-v2-556d6457d-g2rzx              2/2     Running   2 (58m ago) 9d
reviews-v3-564544b4d6-25pjh              2/2     Running   2 (58m ago) 9d

```

Figure 19: Circuit Breaking - Three Concurrent Connections with Circuit Tripped

The results show a dramatic increase in failures: only 36.7% of requests succeed (11 out of 30), with 63.3% failing with HTTP 503 errors. This clearly demonstrates the circuit breaker protecting the service by rejecting excess requests.

```

IP addresses distribution:
10.96.221.209:8000: 11
Code 200 : 11 (55.0 %)
Code 503 : 9 (45.0 %)
Response Header Sizes : count 20 avg 134.75 +/- 121.9 min 0 max 245 sum 2695
Response Body/Total Sizes : count 20 avg 584.75 +/- 310.9 min 241 max 866 sum 11695
All done 20 calls (plus 0 warmup) 2.552 ms avg, 762.1 qps

```

Figure 21: Circuit Breaking - Test Results with 36.7% Success Rate

4.7 Performance Analysis

Analyzing the test results reveals important insights about circuit breaker behavior:

- The circuit breaker effectively limits the number of concurrent connections, preventing resource exhaustion on the backend service.
- Response times remain relatively consistent even under load, indicating that the proxy is managing connections efficiently.
- The outlier detection mechanism quickly identifies failing instances and removes them from the load balancing pool.

```

IP addresses distribution:
10.96.221.209:8000: 18
Code 200 : 13 (43.3 %)
Code 503 : 17 (56.7 %)
Response Header Sizes : count 30 avg 106.16667 +/- 121.4 min 0 max 245 sum 3185
Response Body/Total Sizes : count 30 avg 511.83333 +/- 309.7 min 241 max 866 sum 15355
All done 30 calls (plus 0 warmup) 2.466 ms avg, 1103.4 qps

```

Figure 22: Circuit Breaking - Connection Pool Statistics


```

zgg@LAPTOP-L21QMK1J:~/istio-1.29.2$ kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent request GET stats 2>/dev/nu
ll | grep httpbin | grep pending
cluster.outbound|8000|httpbin.default.svc.cluster.local;.circuit_breakers.default.rq_pending_open: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.circuit_breakers.high.rq_pending_open: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_active: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_failure_eject: 0
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_overflow: 26
cluster.outbound|8000|httpbin.default.svc.cluster.local;.upstream_rq_pending_total: 25

```

Figure 23: Circuit Breaking - Final Status Check and Summary

5. Conclusion

This comprehensive experiment successfully demonstrated four fundamental Istio traffic management features, each addressing critical challenges in microservices architecture:

Traffic Shifting enables gradual, controlled migration between service versions using weighted routing. This approach significantly reduces deployment risk by allowing operators to validate new versions with a small subset of users before full rollout. The experiment showed how traffic can be shifted from 0% to 50% to 100% between versions, providing visual confirmation through the appearance of star ratings.

Request Routing allows dynamic routing based on user identity and custom headers. This capability enables advanced use cases such as A/B testing, feature flagging, and personalized user experiences. The experiment demonstrated how specific users can be routed to different service versions while maintaining default behavior for others.

Fault Injection is a powerful technique for proactively testing system resilience. By injecting delays and abort faults, we were able to identify a critical timeout bug that would have been difficult to detect in normal operation. The experiment highlighted the importance of testing failure scenarios and validating fault tolerance mechanisms before production deployment.

Circuit Breaking prevents cascading failures by limiting concurrent connections and requests to failing services. The experiment demonstrated how circuit breakers protect both clients and services during periods of high load or failure, maintaining overall system stability even when individual components experience issues.

Overall, these experiments showcase Istio's comprehensive traffic management capabilities and demonstrate how they can be leveraged to build resilient, reliable microservices architectures.